

Monte Carlo Estimation of Action Values

From $V(s)$ to $Q(s, a)$: Exploration and ε -Greedy Policies

CSCE 775 – Reinforcement Learning

I. INTRODUCTION: FROM $V(s)$ TO $Q(s, a)$

In the previous guide on Monte Carlo prediction, we learned how to estimate the **state-value function** $V(s)$, which tells us how good it is to *be in* a particular state. This is useful for evaluating a policy, but it does not directly tell us which *action* to take. Knowing that a room is valuable does not tell us which door to walk through to get the most reward.

To actually *choose* actions, we need the **action-value function** $Q(s, a)$, which tells us how good it is to take a specific action a in state s and then follow the policy π thereafter. If we know $Q(s, a)$ for all actions, choosing the best action is trivial: just pick $\arg \max_a Q(s, a)$.

A. Why $V(s)$ Alone Is Not Enough

With only $V(s)$, selecting the best action requires knowing the *model* of the environment. Specifically, we would need to compute the following for each action a .

$$\arg \max_a \sum_{s', r} p(s', r \mid s, a) [r + \gamma V(s')] \quad (1)$$

This requires the transition probabilities $p(s', r \mid s, a)$, which we often do not have. With $Q(s, a)$, we bypass this entirely: we just pick $\arg \max_a Q(s, a)$ without needing any model. This is what makes Q -values essential for **model-free** control.

B. Defining $Q^\pi(s, a)$

The action-value function under policy π is defined as follows.

$$Q^\pi(s, a) = \mathbb{E}_\pi [G_t \mid S_t = s, A_t = a] \quad (2)$$

Let us unpack each piece of notation. $Q^\pi(s, a)$ is the value of taking action a in state s and then following policy π for all subsequent decisions. The symbol $\mathbb{E}_\pi[\cdot]$ means “the expected value (average) if we follow policy π after the first action.” The term G_t is the return (total discounted reward from time t onward). The condition $S_t = s, A_t = a$ means “given that we are in state s at time t and take action a .”

$V(s)$ vs $Q(s)$

Think of $V(s)$ as a review of a restaurant: “This place is great!” It tells you the overall quality, but not what to order. $Q(s, a)$ is like a review of each dish: “The pizza is excellent, but the pasta is mediocre.” With dish-level reviews, you can directly choose the best item on the menu without needing to know anything about the kitchen (the model).

II. THE GRIDWORLD WITH ACTIONS

We reuse the same 3×3 gridworld from the prediction guide. The agent occupies states S_1 through S_9 , where S_9 is the terminal goal state. Stepping into S_9 yields $R = +10$, and every other step yields $R = -1$. The discount factor is $\gamma = 0.9$. The key difference now is that we explicitly label the agent's **actions**: Up ($\uparrow$), Down ($\downarrow$), Left ($\leftarrow$), and Right ($\rightarrow$).

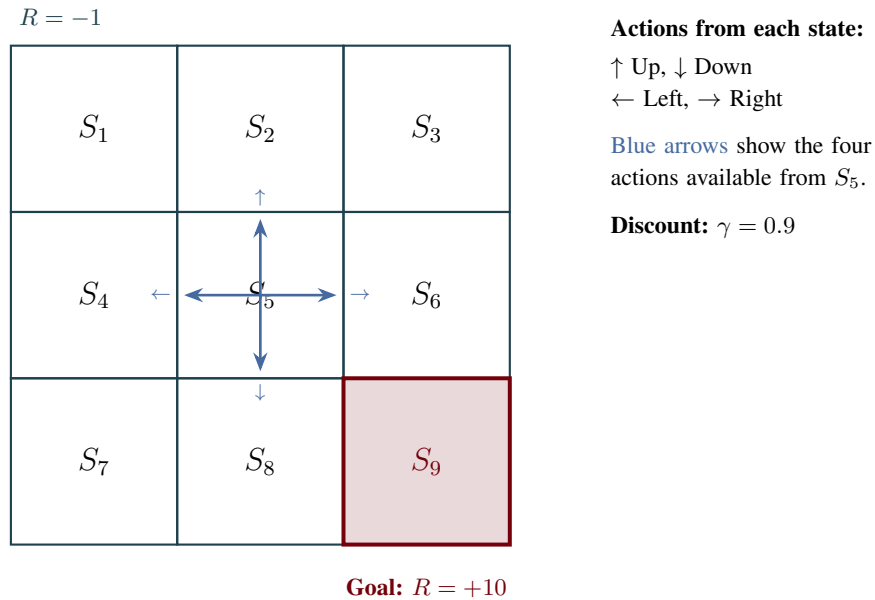


Fig. 1. The 3×3 gridworld with actions. Each non-terminal state has four actions. Blue arrows illustrate the action set from S_5 .

III. A SAMPLE EPISODE WITH ACTIONS

Consider the same episode from the prediction guide, but now we explicitly record the action taken at each step.

Episode Trajectory with Actions

$S_1 \xrightarrow{R} S_2 \xrightarrow{D} S_5 \xrightarrow{L} S_4 \xrightarrow{U} S_1 \xrightarrow{R} S_2 \xrightarrow{R} S_3 \xrightarrow{D} S_6 \xrightarrow{D} S_9$

R = Right, D = Down, L = Left, U = Up.

States in **rose** are revisits. Notice that (S_1, Right) and (S_2, Right) are repeated (state, action) pairs, while (S_2, Down) at $t = 1$ is a *different* (state, action) pair from (S_2, Right) at $t = 5$.

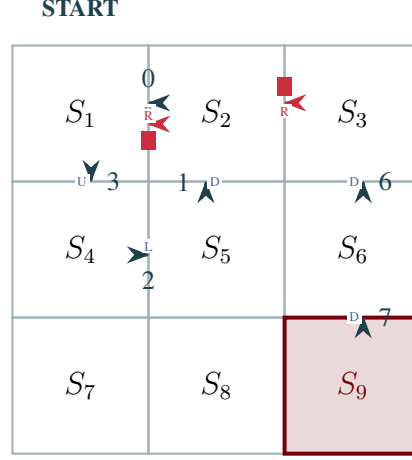


Fig. 2. Episode path with action labels. Each arrow shows the time step and the action taken. Rose arrows indicate revisited (state, action) pairs.

IV. COMPUTING RETURNS FOR (STATE, ACTION) PAIRS

The return computation is identical to the prediction guide. We compute G_t backward from the terminal state using $G_t = R_{t+1} + \gamma G_{t+1}$. The difference is that each return is now associated with a **(state, action) pair** rather than just a state.

Backward Return Computation

$G_7 = R_8 = +10$	$(S_6, D) \rightarrow S_9$
$G_6 = R_7 + \gamma G_7 = -1 + 0.9(10) = 8.0$	(S_3, D)
$G_5 = R_6 + \gamma G_6 = -1 + 0.9(8.0) = 6.2$	$(S_2, R) - 2\text{nd visit}$
$G_4 = R_5 + \gamma G_5 = -1 + 0.9(6.2) = 4.58$	$(S_1, R) - 2\text{nd visit}$
$G_3 = R_4 + \gamma G_4 = -1 + 0.9(4.58) = 3.122$	(S_4, U)
$G_2 = R_3 + \gamma G_3 = -1 + 0.9(3.122) = 1.810$	(S_5, L)
$G_1 = R_2 + \gamma G_2 = -1 + 0.9(1.810) = 0.629$	(S_2, D)
$G_0 = R_1 + \gamma G_1 = -1 + 0.9(0.629) = -0.434$	$(S_1, R) - 1\text{st visit}$

TABLE I
RETURNS G_t FOR EACH (STATE, ACTION) PAIR IN THE EPISODE.

Time t	State	Action	Return G_t	Visit #
0	S_1	Right	-0.434	1st
1	S_2	Down	0.629	1st
2	S_5	Left	1.810	1st
3	S_4	Up	3.122	1st
4	S_1	Right	4.58	2nd
5	S_2	Right	6.2	2nd
6	S_3	Down	8.0	1st
7	S_6	Down	10.0	1st

Notice the critical distinction: (S_2, Down) at $t = 1$ and (S_2, Right) at $t = 5$ are *different* (state, action) pairs, even though they share the same state. The pair (S_1, Right) appears twice (at $t = 0$ and $t = 4$), and (S_2, Right) also appears twice if we count

$t = 5$ as a revisit. Meanwhile, (S_2, Down) appears only once. This is the key difference from state-value estimation: the same state can appear with different actions, and each (state, action) combination is tracked independently.

V. FIRST-VISIT MC FOR ACTION VALUES

Definition: First-Visit MC for $Q(s)$

For each (state, action) pair (s, a) appearing in an episode, record the return G_t only for the **first time** that specific pair (s, a) is visited. The action-value estimate is the average of these first-visit returns across all episodes.

$$Q(s, a) \approx \frac{1}{N(s, a)} \sum_{i=1}^{N(s, a)} G_t^{(i)} \quad \text{where each } G_t^{(i)} \text{ is from the first visit to } (s, a) \text{ in episode } i \quad (3)$$

The key distinction from the state-value version is that we track *(state, action) pairs*, not just states. Two visits to the same state with different actions count as visits to different pairs. In our episode, (S_1, Right) is visited at $t = 0$ and $t = 4$. First-visit MC uses only the return from $t = 0$. Meanwhile, (S_2, Down) at $t = 1$ and (S_2, Right) at $t = 5$ are different pairs entirely, so both are first visits.

After processing this single episode, our Q -table looks like the following.

TABLE II
Q-TABLE AFTER ONE EPISODE (FIRST-VISIT MC). ONLY OBSERVED (STATE, ACTION) PAIRS HAVE ESTIMATES.

State	$Q(s, \text{Up})$	$Q(s, \text{Down})$	$Q(s, \text{Left})$	$Q(s, \text{Right})$
S_1	—	—	—	-0.434
S_2	—	0.629	—	6.2
S_3	—	8.0	—	—
S_4	3.122	—	—	—
S_5	—	—	1.810	—
S_6	—	10.0	—	—

The dashes (—) represent (state, action) pairs that were never visited. We have *no estimate* for these pairs. This is a serious problem that we will address in Section VII.

First-Visit MC for $Q(s)$

$Q(s, a) \leftarrow 0$, $\text{Returns}(s, a) \leftarrow$ empty list, for all s, a

Generate episode: $S_0, A_0, R_1, S_1, A_1, R_2, \dots, S_T$

$G \leftarrow 0$

$t = T - 1, T - 2, \dots, 0$:

$G \leftarrow R_{t+1} + \gamma G$

(S_t, A_t) not in $\{(S_0, A_0), (S_1, A_1), \dots, (S_{t-1}, A_{t-1})\}$:

◁ first visit only

Append G to $\text{Returns}(S_t, A_t)$

$Q(S_t, A_t) \leftarrow \text{average}(\text{Returns}(S_t, A_t))$

VI. Q-TABLE VISUALIZATION

The following diagram shows the Q -table as a grid where each cell is subdivided into four quadrants, one for each action. This visualization makes it easy to see which (state, action) pairs have been estimated and which remain unknown.

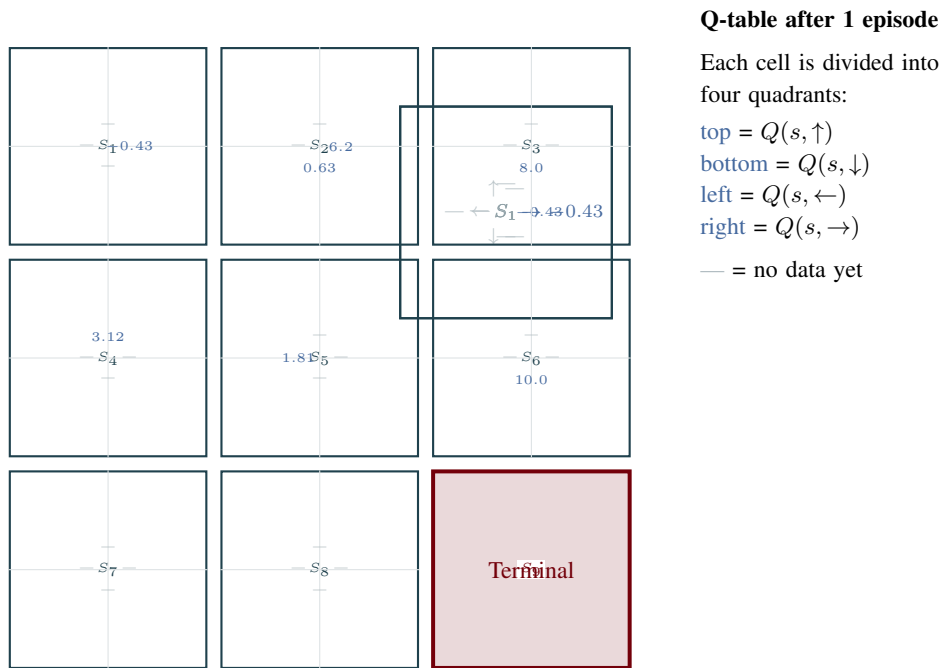


Fig. 3. Q-table visualization. Each cell shows estimated action values in four quadrants. Most (state, action) pairs remain unestimated (—) after a single episode.

VII. THE EXPLORATION PROBLEM

Look at Table II and Figure 3 again. After one episode, we have estimates for only 8 out of 32 possible (state, action) pairs ($8 \text{ non-terminal states} \times 4 \text{ actions} = 32 \text{ pairs}$). The other 24 pairs have *no data at all*. This is the **exploration problem**, and it is the central challenge of estimating action values with Monte Carlo methods.

A. Why This Problem Didn't Exist for $V(s)$

When estimating $V(s)$, every visit to state s gives us useful data, regardless of which action the agent took. If the agent visits S_2 three times (taking Down, Right, and Left), all three visits contribute returns for estimating $V(S_2)$.

With $Q(s, a)$, things are different. Each visit to S_2 only provides data for the specific action taken. If the policy always chooses Right from S_2 , we accumulate returns for $Q(S_2, \text{Right})$ but *never* see $Q(S_2, \text{Up})$, $Q(S_2, \text{Down})$, or $Q(S_2, \text{Left})$. We cannot improve the policy because we do not know if those unexplored actions might actually be better.

B. The Burger Place Analogy

Imagine you always eat at the same burger restaurant. You know exactly how good the burgers are (you have a great estimate of $Q(\text{dinner, burgers})$), but you have no idea about the pizza place next door. Maybe the pizza is far better, maybe it is worse. You will never know unless you *explore* by trying it. But if you always follow your current “policy” of choosing burgers, you are stuck with a potentially suboptimal choice forever.

This is exactly the exploration problem in MC estimation. A deterministic policy (one that always picks the same action in a given state) will never explore alternative actions, and thus can never discover if a different action is better.

The Exploration Problem

When estimating $Q(s, a)$, a deterministic policy leaves many (state, action) pairs unvisited. Without data for these pairs, we cannot compare actions and cannot improve the policy. We need mechanisms to ensure that *all* (state, action) pairs are visited sufficiently often. The two main solutions are **exploring starts** and **ϵ -greedy policies**.

VIII. SOLUTION 1: EXPLORING STARTS

Definition: Exploring Starts

Exploring starts requires that every (state, action) pair has a nonzero probability of being selected as the starting state and action of an episode. At the beginning of each episode, we randomly choose both a starting state S_0 and a starting action A_0 , then follow the policy π for all subsequent steps.

The idea is simple: if we cannot guarantee that the policy will explore all (state, action) pairs during an episode, we can at least guarantee that every pair gets to be the *first* step of some episode. Over many episodes, each pair will eventually be sampled as the starting pair, giving us at least one return for every $Q(s, a)$.

A. How It Works

At the start of each episode, uniformly sample a state S_0 from all non-terminal states and an action A_0 from all available actions. Take action A_0 , observe the reward and next state, and then follow the current policy π for all remaining steps until the episode terminates. Since we randomly select the starting pair, all 32 (state, action) pairs in our gridworld will eventually be sampled.

B. Limitations

Exploring starts is a useful theoretical tool, but it has a major practical limitation: it requires the ability to start an episode from *any* state-action pair. In many real-world settings, this is impossible. You cannot restart a self-driving car in the middle of a highway facing the wrong direction. You cannot begin a chess game from an arbitrary board position. You cannot place a medical robot in the middle of a surgical procedure. Exploring starts assumes a level of control over the environment that we typically do not have.

MC with Exploring Starts (MC ES) Pseudocode

$Q(s, a) \leftarrow$ arbitrary, Returns(s, a) $\leftarrow$ empty list, for all s, a
 $\pi(s) \leftarrow$ arbitrary deterministic policy

Choose $S_0 \sim \text{Uniform}(\mathcal{S})$ and $A_0 \sim \text{Uniform}(\mathcal{A}(S_0))$

◁ exploring start

Generate episode from S_0, A_0 following π : $S_0, A_0, R_1, S_1, A_1, \dots, S_T$

$G \leftarrow 0$

$t = T - 1, T - 2, \dots, 0$:

$G \leftarrow R_{t+1} + \gamma G$

(S_t, A_t) not in $\{(S_0, A_0), \dots, (S_{t-1}, A_{t-1})\}$:

Append G to Returns(S_t, A_t)

$Q(S_t, A_t) \leftarrow \text{average}(\text{Returns}(S_t, A_t))$

$\pi(S_t) \leftarrow \arg \max_a Q(S_t, a)$

◁ policy improvement

IX. SOLUTION 2: ε -GREEDY POLICIES**Definition: ε -Greedy Policy**

An **ε -greedy policy** selects the greedy action (the one with the highest Q -value) with probability $1 - \varepsilon + \frac{\varepsilon}{|\mathcal{A}(s)|}$, and selects each of the other actions with probability $\frac{\varepsilon}{|\mathcal{A}(s)|}$. This ensures every action in every state has at least a $\frac{\varepsilon}{|\mathcal{A}(s)|}$ probability of being chosen.

$$\pi(a | s) = \begin{cases} 1 - \varepsilon + \frac{\varepsilon}{|\mathcal{A}(s)|} & \text{if } a = \arg \max_{a'} Q(s, a') \\ \frac{\varepsilon}{|\mathcal{A}(s)|} & \text{otherwise} \end{cases} \quad (4)$$

The ε -greedy policy is the most practical solution to the exploration problem. Instead of requiring special starting conditions, it builds exploration directly into the policy itself. At every step, there is a small chance of taking a random action, which means every (state, action) pair has a nonzero probability of being visited during normal operation.

A. Worked Example

Suppose $\varepsilon = 0.1$ and there are $|\mathcal{A}| = 4$ actions (Up, Down, Left, Right). The greedy action (say, Right) gets probability $1 - 0.1 + \frac{0.1}{4} = 0.925$. Each of the three non-greedy actions gets probability $\frac{0.1}{4} = 0.025$.

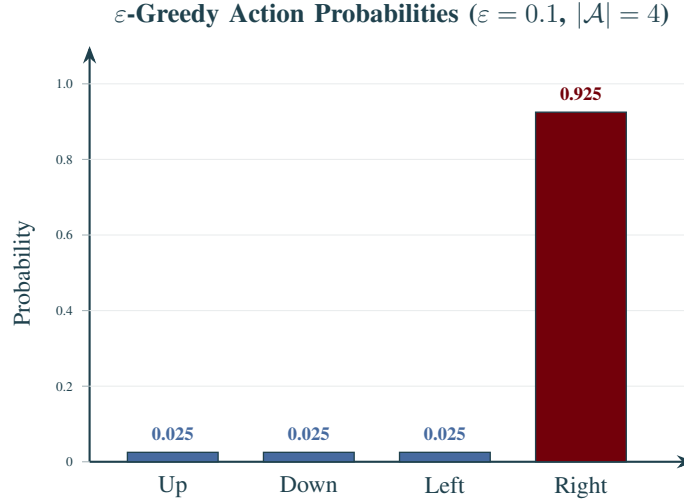


Fig. 4. Probability distribution over actions for an ε -greedy policy with $\varepsilon = 0.1$. The greedy action gets 92.5% of the probability mass, while each non-greedy action gets 2.5%.

Even though the non-greedy probabilities are small (2.5% each), they guarantee that over many episodes, every action in every state will be tried. This solves the exploration problem without requiring the ability to control the starting state.

B. The Exploration–Exploitation Trade-Off

The parameter ε controls the balance between **exploration** (trying new actions to gather information) and **exploitation** (choosing the action we currently believe is best). A large ε (e.g., 0.5) explores aggressively but makes many suboptimal choices. A small ε (e.g., 0.01) mostly exploits the current best action but explores slowly. A common practice is to start with a larger ε and gradually decay it over time, allowing the agent to explore broadly at first and then settle into a more refined policy.

Choose ε -Greedy When...

- You *cannot control the starting conditions* of episodes. This is the most common real-world scenario and is the primary motivation for ε -greedy over exploring starts.
- You want a *simple, practical* exploration strategy that works in any environment.
- You are doing *on-policy* learning, where the policy you are evaluating is the same policy generating the data. The ε -greedy policy generates its own exploratory data while still mostly exploiting what it has learned.

X. PUTTING IT ALL TOGETHER: MC CONTROL PREVIEW

We have now seen how to estimate action values $Q(s, a)$ from episodes and how to ensure all (state, action) pairs are explored. The natural next step is **Monte Carlo control**: using Q -value estimates to actually *improve* the policy.

The MC control loop works as follows. First, generate an episode using the current policy (with exploration, via exploring starts or ε -greedy). Second, compute the returns and update the Q -table using first-visit or every-visit MC. Third, improve the policy by making it greedy (or ε -greedy) with respect to the updated Q -values: for each state s , set $\pi(s) = \arg \max_a Q(s, a)$.

This cycle of *evaluation* (estimating Q) and *improvement* (updating π) is called **generalized policy iteration (GPI)**. Each iteration brings the policy closer to optimal, and with sufficient episodes, the process converges to the optimal policy π^* and optimal action-value function Q^* .

Foundation for MC Control

Estimating $Q(s, a)$ is the essential building block of MC control methods. Without accurate Q -value estimates, we cannot compare actions and thus cannot improve the policy. The two exploration strategies (exploring starts and ε -greedy) ensure we get the data we need for every (state, action) pair. MC control methods, including on-policy and off-policy variants, will be covered in depth in a future guide.

XI. KEY TAKEAWAYS

Summary: From $V(s)$ to $Q(s)$

$V(s)$ vs $Q(s, a)$:

- $V(s)$ tells us how good a state is, but choosing actions from $V(s)$ requires a model of the environment.
- $Q(s, a)$ tells us how good each action is in each state, enabling *model-free* action selection via $\arg \max_a Q(s, a)$.
- MC estimation of $Q(s, a)$ works the same as for $V(s)$, except we track returns for (state, action) pairs instead of just states.

Why exploration matters:

- A deterministic policy only visits a fraction of (state, action) pairs, leaving most Q -values unestimated.
- Without estimates for all actions, we cannot compare them and thus cannot improve the policy.

Exploring starts vs ε -greedy:

- *Exploring starts* randomly selects the initial (state, action) pair for each episode. It guarantees coverage but requires the ability to control starting conditions, which is often impractical.
- *ε -greedy policies* mix exploitation (choosing the best known action with probability $1 - \varepsilon + \varepsilon/|\mathcal{A}|$) with exploration (choosing a random action with probability ε). This is the practical default for ensuring exploration.